



Privacy and Anonymity

Práctica #3: Cifrado Homomórfico

En esta práctica, construiremos un sistema de cifrado homomórfico de juguete. El tamaño del vector secreto (n en nuestras diapositivas) será un parámetro, pero lo mantendremos relativamente pequeño por razones computacionales. Lo primero que necesitamos es generar el error. Recuerda de las clases que la distribución debe ser tal que la probabilidad de que $e = x \in \mathbb{Z}$ debe ser proporcional a

$$\exp\left(-\frac{|x|^2}{2\alpha^2 q^2}\right)$$

para algunos parámetros α y q . Dado que no es tan fácil muestrear esta distribución, una forma aproximada de hacerlo es muestrear a partir de una distribución Gaussiana con media cero y varianza $\alpha^2 q^2$ (o desviación estándar αq) y redondear el resultado al entero más cercano, finalmente reduciéndolo módulo q . Para mayor comodidad, como discutimos en las clases, la reducción módulo q produce un resultado en el rango $[-q/2, q/2 - 1]$, suponiendo que q es un número par. Formalmente, sea x una muestra de una Gaussiana con media cero y varianza $\alpha^2 q^2$, entonces la muestra de salida es $\lfloor x \rfloor \bmod q \in [-q/2, q/2 - 1]$.

Q1 Escribe una función `generate_error(m, alpha, q)` que devuelva un vector con m muestras tomadas independientemente de una distribución Gaussiana con desviación estándar $\alpha \cdot q$, redondeadas al entero más cercano y reducidas módulo q . Para $q = 100$, genera $m = 10000$ muestras de error y representa su histograma (recuerda que las salidas son números enteros, por lo que debes ajustar los “bins” del histograma en consecuencia) para los siguientes tres casos: a) $\alpha = 0.01$; b) $\alpha = 0.1$; c) $\alpha = 1$. Discute los resultados.

Estamos listos para implementar un sistema de cifrado homomórfico (de juguete). Excepto por la última pregunta (de puntuación extra) estableceremos los siguientes parámetros: $n = 256$, $q = 2^{16}$, $\Delta = 2^{10}$. El secreto se elegirá de manera que cada muestra (recuerda que es un vector de longitud n) se seleccione con probabilidad uniforme del conjunto $\{-1, 0, 1\}$. Todos los vectores $\mathbf{a}$ que necesitaremos se generarán tomando muestras uniformes de $\mathbb{Z}_q^n$.

Q2 Escribe un script que calcule para cada valor de $\alpha \in \{0.5\text{e-}03, 1\text{e-}03, 1.5\text{e-}03, 2.0\text{e-}03, \dots, 4.5\text{e-}03\}$ (en Python: `np.arange(5e-04, 5e-03, 5e-04)`) lo siguiente:

1. Repite 1000 veces:
2. Genera un nuevo vector secreto aleatorio $\mathbf{s}$ y un nuevo vector aleatorio $\mathbf{a}$.
3. Para el mensaje $m = 10$, cifra $\Delta \cdot m$, es decir, genera la cifra de salida $(\mathbf{a}, b)$.
4. Descifra $(\mathbf{a}, b)$ usando $\mathbf{s}$ y extrae el mensaje (llama al resultado $\hat{m}$).
5. Si $\hat{m} \neq m$ entonces suma 1 al contador de errores.
6. Fin

Observa que el contador de errores será 0 si no se produjeron errores de descifrado y 1000 si cada mensaje fue descifrado incorrectamente. Haz una tabla que muestre el contador de errores para cada uno de los posibles valores de α . Discute los resultados. Incluye el código de tu script como parte de

tu respuesta.

Q3 Escribe un script que calcule para cada valor de $\alpha \in \{0.5\text{e-}03, 1\text{e-}03, 1.5\text{e-}03, 2.0\text{e-}03, \dots, 4.5\text{e-}03\}$ (en Python: `np.arange(5e-04, 5e-03, 5e-04)`) lo siguiente:

1. Repite 1000 veces:
2. Genera un nuevo vector secreto aleatorio $\mathbf{s}$ y dos nuevos vectores aleatorios $\mathbf{a}_1$ y $\mathbf{a}_2$.
3. Para el mensaje $m_1 = 10$, cifra $\Delta \cdot m_1$, es decir, genera la cifra de salida $(\mathbf{a}_1, b_1)$. Haz lo mismo con el mensaje $m_2 = 8$, generando el ciphertext de salida $(\mathbf{a}_2, b_2)$.
4. Opera (sin usar $\mathbf{s}$) con $(\mathbf{a}_1, b_1)$ y $(\mathbf{a}_2, b_2)$ para generar un cifrado de $\Delta(m_1 + m_2)$.
5. Descifra el resultado del paso anterior usando $\mathbf{s}$ y extrae el mensaje (llama al resultado $\hat{m}$).
6. Si $\hat{m} \neq (m_1 + m_2)$ entonces suma 1 al contador de errores.
7. Fin

Haz una tabla que muestre el contador de errores para cada uno de los posibles valores de α . Discute los resultados, también en comparación con los resultados en la pregunta Q2. Incluye el código de tu script como parte de tu respuesta.

Q4 Escribe un script que calcule para cada valor de $\alpha \in \{0.5\text{e-}03, 1\text{e-}03, 1.5\text{e-}03, 2.0\text{e-}03, \dots, 4.5\text{e-}03\}$ (en Python: `np.arange(5e-04, 5e-03, 5e-04)`) lo siguiente:

1. Repite 1000 veces:
2. Genera un nuevo vector secreto aleatorio $\mathbf{s}$ y un nuevo vector aleatorio $\mathbf{a}$.
3. Para el mensaje $m = 3$, cifra $\Delta \cdot m$, es decir, genera la cifra de salida $(\mathbf{a}, b)$.
4. Sin usar $\mathbf{s}$, opera con $(\mathbf{a}, b)$ para generar un cifrado de $c \cdot \Delta \cdot m$, donde $c = 4$.
5. Descifra el resultado del paso anterior usando $\mathbf{s}$ y extrae el mensaje (llama al resultado $\hat{m}$).
6. Si $\hat{m} \neq c \cdot m$, entonces suma 1 al contador de errores.
7. Fin

Haz una tabla que muestre el contador de errores para cada uno de los posibles valores de α . Discute los resultados, también en comparación con los resultados en la pregunta Q3. Incluye el código de tu script como parte de tu respuesta.

Q5 En esta pregunta, se te pide hacer una multiplicación como en la pregunta anterior, pero esta vez $c = 10$. Primero, discute qué sucede si ejecutas el script en Q4 usando $c = 10$. A continuación, se te pide utilizar la *descomposición gadget* para hacer la multiplicación con datos cifrados. Para la descomposición gadget de c usa $p = 3$, es decir, escribe c como $c^{(0)} + c^{(1)}2^p$, donde debes encontrar los valores $c^{(0)}, c^{(1)}$ sabiendo que deben estar en el intervalo $[-\frac{2^p}{2}, \frac{2^p}{2} - 1]$. Haz una tabla que muestre el contador de errores para cada uno de los posibles valores de α . Discute los resultados en comparación con lo que obtuviste realizando la multiplicación sin la descomposición gadget. Incluye el código de tu script como parte de tu respuesta.

Q6. Puntuación extra En esta pregunta, cambiamos los parámetros utilizados anteriormente a $q = 2^{24}$, $n = 128$, $\Delta = 2^{18}$. Escribe un script que para $\alpha = 1.0\text{e-}05$ calcule lo siguiente:

1. Repite 1000 veces:
2. Genera dos nuevos vectores secretos aleatorios $\mathbf{s}$ y $\mathbf{s}'$ y un nuevo vector aleatorio $\mathbf{a}$.

3. Para el mensaje $m = 10$, cifra $\Delta \cdot m$, es decir, genera la cifra de salida $(\mathbf{a}, b)$.
4. Considerando las descomposiciones gadget con $p = 4$, genera todos los cifrados necesarios para llevar a cabo un cambio de clave de $\mathbf{s}$ a $\mathbf{s}'$.
5. Realiza el cambio de clave con datos cifrados (obviamente, descifrar el mensaje con $\mathbf{s}$ y volver a cifrarlo con $\mathbf{s}'$ no está permitido).
6. Descifra el resultado del paso anterior usando $\mathbf{s}'$ y extrae el mensaje (llama al resultado $\hat{m}$).
7. Si $\hat{m} \neq c \cdot m$ entonces suma 1 al contador de errores.
8. Fin

Si todo salió bien, el contador de errores debería ser cero. Incluye el código de tu script en tu respuesta.